Stronger Encoders and Structure-Constrained Decoding for DiCo-NLI (Track 1)

Muhammad Muneeb Baig, Syed Kamran Ali Shah, Kashaf Ali Haroon
MS Data Science, SEECS, NUST
CS-878 Deep Learning, Assignment 2: Proposed Approach and Experimental Design
Code and results: `https://github.com/MuneebBaig71/DiCo-NLI_SemEval`
November 2026

Abstract

In Assignment 1 (A1), a fine-tuned RoBERTa-base reached 0.765 weighted F1 (W-F1) and 0.739 HardCons on the DiCo-NLI English development set, and most of its errors were on the NEG boundary. We propose a pair-aware system: a fully fine-tuned DeBERTa-v3 cross-encoder that sees both orientations of a pair in the same batch, an optional reversal-consistency loss, NEG class weighting, and decoding rules that label both orientations together. Over three seeds, DeBERTa-v3 raises argmax W-F1 from 0.758 to 0.821 and HardCons from 0.759 to 0.839, both significant under a paired bootstrap. Symmetric decoding adds 2.3 HardCons points with no extra training. The largest gain, to 0.925 W-F1, comes from a property of the released data (rows with a reversed partner are never NEG) and is specific to this dataset. The consistency loss has no significant effect. The development set also shaped our design, so we treat these as verification results and make no final performance claims.

1 Proposed Approach

Task. The input is an ordered pair of short English phrases (t_1, t_2), and the output is one of four labels for that ordered pair: EQ, FWD (t_1 entails t_2), BWD (t_2 entails t_1) or NEG, which merges opposition, similarity, relatedness and no relation (DiCo-NLI Organizers, 2026; MacCartney, 2009). The prediction target is one label per row. When a pair appears in both orientations, a correct system must also satisfy $f(t_2, t_1) = \mathrm{rev}(f(t_1, t_2))$, with $\mathrm{rev}(\text{FWD}) = \text{BWD}$, $\mathrm{rev}(\text{BWD}) = \text{FWD}$ and $\mathrm{rev}(\text{EQ}) = \text{EQ}$. We therefore want each row right and the two orientations coherent.

System. The system has four parts (Figure 2). Preprocessing groups the two orientations of a pair by `pair_id`. A fully fine-tuned DeBERTa-v3-base cross-encoder scores each orientation. Training uses class-weighted cross-entropy plus an optional consistency term. A pair-aware decoder then labels both orientations together.

Role of the A1 features. A1's most useful handcrafted features were directional: lengths, how much of one phrase the other covers, and containment. A cross-encoder reads both phrases together and can learn these signals from the tokens, so we do not feed them in. They stay in the classical baseline, which shows what the encoder adds beyond them.

Data decisions. We do no text normalization; the tokenizers are cased. Each row becomes "[CLS] t_1 [SEP] t_2 [SEP]", truncated at 64 tokens, and the longest dev pair has 18 words, so the limit does not bind. We do not augment with reversed pairs: 1,282 of the 1,760 training pairs already appear in both orientations, and reversing the 478 single NEG rows would create NEG rows with partners, which never occur in the data. Batches are built from whole pairs, so both orientations of a pair always share a batch. We do no feature selection.

Variation and noise. NEG is the smallest class (16% of train) and the hardest, so we weight it in the loss and report Macro-F1. Some gold labels are debatable, such as near-paraphrases labelled NEG in A1; we do not model label noise but plan an audit. Inputs are short, so length is not a concern. The system covers Track 1 only; a multilingual pilot is planned. Domain, speaker and multimodal variation do not apply.

Expected limitations. We expect errors on the EQ/FWD/BWD boundary for near-paraphrases, where a small wording change may or may not change entailment, and on the mixed NEG class. The structural rules depend on how the dataset was built, and the training set is small (2,738 rows after the validation split).

2 Method Justification

Table 1 links each design decision to evidence from A1. For the three main decisions we give the observation, our interpretation and the modelling implication.

Encoder. *Observation:* A1's errors concentrate on EQ versus NEG for lexically similar phrases. *Interpretation:* the model has to judge whether a small wording change alters the meaning, which depends on word order and relative position. *Implication:* DeBERTa-v3 encodes content and relative position separately and was the strongest model in the organisers' pilot (He et al., 2023; Apaolaza Larraya et al., 2026). We test whether it draws this boundary better and do not assume it.

Joint decisions. *Observation:* RoBERTa almost never swaps FWD and BWD, yet still loses SoftCons and HardCons on pairs where it calls one orientation NEG. *Interpretation:* the two orientations are scored independently, so nothing makes their decisions agree. *Implication:* decide at pair level, with a consistency regulariser in training, following logic-constrained NLI training (Minervini and Riedel, 2018; Li et al., 2019), and with symmetric decoding at inference. Pretrained models are often inconsistent under such transformations (Jang et al., 2022).

Pair structure. *Observation:* in train, all 478 single rows are NEG and none of the 2,564 rows in the 1,282 pairs is; dev shows the same pattern (277 pairs, 106 single NEG rows; Figure 1). *Interpretation:* this is a construction

Evidence from A1 or the data	Design decision	Expected effect
NEG is the weakest class (RoBERTa F1 0.565) and the smallest (16% of train)	NEG class weight $w_{\text{NEG}} = 2$ in the loss; report Macro-F1 and per-class recall	Higher NEG recall; less dominance of the three larger classes
69 near-paraphrases rejected as NEG; EQ F1 only 0.725	Stronger encoder: DeBERTa-v3-base (He et al., 2023)	Better separation of EQ from NEG on lexically similar pairs
FWD/BWD confused only twice in each direction, but 15–19 pairs per seed are NEG in exactly one orientation	Decide both orientations jointly: pair-aware batching, consistency loss, symmetric decoding	Remove one-sided errors; raise HardCons without hurting W-F1
Length-only LR: SoftCons 0.989 but W-F1 0.464	Judge every change on W-F1 and HardCons together, never SoftCons alone	Avoid rewarding systems that are consistent but wrong
In train and dev, every row with a reversed partner is non-NEG and every row without one is NEG	Structure-constrained decoding, reported separately from model gains	Large NEG gain on dev; transfers to test only if the test file is built the same way

Table 1: Evidence from A1, the design decision it motivates, and the expected effect.

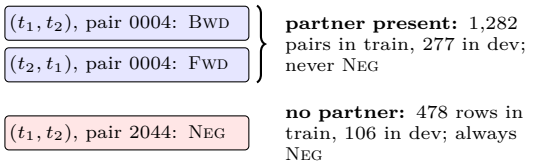


Figure 1: How the released data are built. Rows that share a `pair_id` are the two orientations of one pair, and their labels follow the reversal rule; a row without a partner is always NEG. Partners are visible in the participant file, so the decoding rules can use this at test time without gold labels.

artifact, like the annotation artifacts found in other NLI datasets (Gururangan et al., 2018). Partners can be found from `pair_id` in the participant file, which is available at test time without any gold label. *Implication:* decoding rules may use the structure, but we report them separately as dataset-specific. The structure also explains why 9 of the other 25 English submissions on the development leaderboard (8 November) report SoftCons of exactly 1.0, since SoftCons fails any pair where either prediction is NEG.

3 System Architecture

Encoder and head. We load `microsoft/deberta-v3-base` (about 184M parameters) through `AutoModelForSequenceClassification`. The head is the DeBERTa pooler (first token, dense layer, GELU) followed by a linear layer to four logits. It starts from random weights, and dropout is 0.1. All parameters are trained and nothing is frozen. RoBERTa uses the same setup with its own head.

Training objective. Let p_{12} and p_{21} be the softmax outputs for (t_1, t_2) and (t_2, t_1), and let π swap the FWD and BWD probabilities while keeping EQ and NEG. For each reversible pair in a batch the consistency term is

$$\mathcal{L}_{\text{cons}} = \tfrac{1}{2}\Big[\mathrm{KL}\big(p_{12} \,\|\, \pi(p_{21})\big) + \mathrm{KL}\big(\pi(p_{21}) \,\|\, p_{12}\big)\Big], \quad (1)$$

and the batch loss is

$$\mathcal{L} = \mathrm{CE}_w(y, p) + \lambda\, \overline{\mathcal{L}_{\text{cons}}}, \quad (2)$$

where CE_w is cross-entropy over all rows with weight w_{NEG} on NEG and 1 elsewhere. Single rows have no partner and get only CE_w.

Decoding rules. These only change how labels are chosen, so they apply to any trained model. *argmax* takes each row's top label, as in A1. *symmetric* averages a row's probabilities with its partner's swapped probabilities, $\tilde{p}_{12} = \frac{1}{2}\big(p_{12} + \pi(p_{21})\big)$, so the pair is coherent by construction. *struct_pair* also forbids NEG on rows with a partner, and *struct_full* additionally labels every row without a partner NEG. struct_full only changes single rows, which never enter SoftCons or HardCons, so it can raise W-F1 but not HardCons.

4 Comparison Systems

All systems use the same dev set and the official scorer (Table 2).

Simple baselines. The majority class (FWD) gets 0.288 accuracy and 0.129 W-F1. No label dominates: FWD, BWD and EQ each cover 26–29% of dev and NEG 16%, so the task is only mildly imbalanced, with NEG as the minority. A length-only logistic regression tests the heuristic that the longer phrase entails the shorter. It is highly consistent (SoftCons 0.989) but mostly wrong (W-F1 0.464), which is why we never compare systems on SoftCons alone.

Classical baselines. Logistic regression (L2, $C = 1$, L-BFGS, no class weighting, no resampling), trained on the full training set in A1, on three inputs: 26 standardised engineered features (lengths, coverage and containment in both directions, extra words, overlap, negation and number mismatch, and part-of-speech counts of non-shared words); TF-IDF unigrams and bigrams with separate blocks for t_1 and t_2, which keeps direction; and both together. All 26 features are used, with no feature selection: they come from the A1 analysis, they are few relative to the 3,042 training rows, and L2 regularisation already down-weights weak ones.

Pretrained baselines. BERT-base and RoBERTa-base were fully fine-tuned in A1 as cross-encoders with a linear head on the first token (lr 3×10^{-5}, batch 16, 6 epochs, final epoch kept) (Devlin et al., 2019; Liu et al., 2019). M0 retrains RoBERTa-base under the A2 protocol for a like-for-like comparison.

Proposed systems. M1 is DeBERTa-v3-base with cross-entropy only. M2 adds the consistency loss ($\lambda = 1$), and M2b uses $\lambda = 0.5$ on one seed. M3 adds NEG weighting ($w_{\text{NEG}} = 2$) to M2. Each is evaluated under all four decoding rules and as a three-seed probability ensemble. What sets them apart from the baselines is pair-

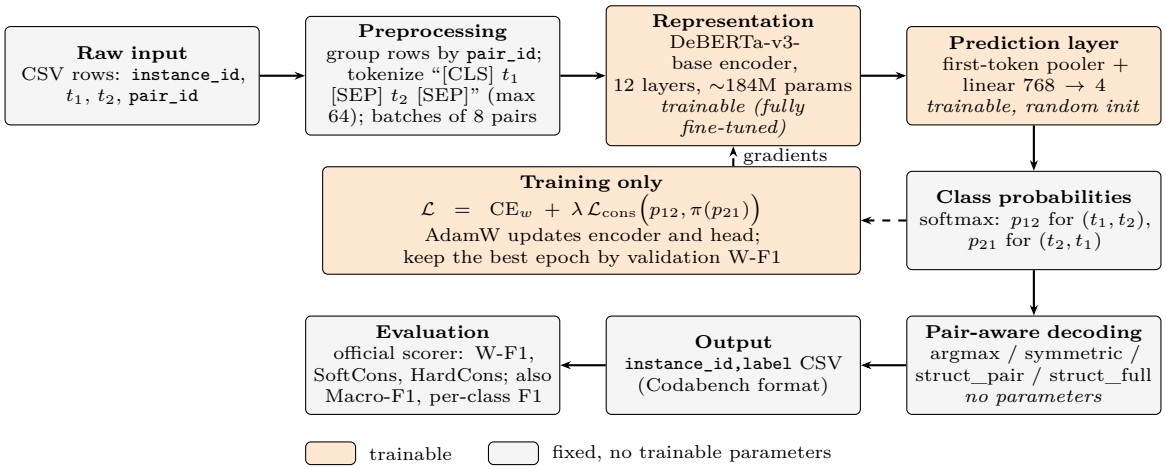


Figure 2: Proposed system. Both orientations of a pair pass through the same encoder and head in the same batch. Nothing is frozen: all encoder and head parameters are updated. π swaps the FWD and BWD probabilities. Single (NEG) rows receive only the cross-entropy term. The decoding rules and the scorer have no trainable parameters.

System (argmax)	W-F1	Soft	Hard
Majority class	0.129	0.000	0.000
LR, length only	0.464	0.989	0.628
LR, engineered features	0.593	0.877	0.653
LR, TF-IDF + features	0.620	0.856	0.661
BERT-base (A1)	0.705	0.774	0.693
RoBERTa-base (A1)	0.765	0.804	0.739
M0 RoBERTa, A2 protocol	0.758	0.819	0.759
M1 DeBERTa-v3	0.821	0.892	0.839
M3 + cons. + NEG wt.	0.827	0.878	0.828

Table 2: Comparison systems on dev (official scorer). Soft/Hard: SoftCons/HardCons. A1 rows are from A1; neural rows are three-seed means. Results under the other decoding rules are in Table 6.

level treatment of the task, in training and at inference.

5 Ablation Plan

Each ablation changes one component and answers one research question (Table 3). A to F were run here as preliminary checks; G to L, including the frozen-versus-fine-tuned comparison, are planned for A3. Significance is judged with the bootstrap in Section 6.

6 Evaluation Measures

Official metrics. All three range from 0 to 1, and higher is better. For class c with precision P_c, recall R_c and support n_c among N rows,

$$\mathrm{F1}_c = \frac{2P_cR_c}{P_c + R_c}, \qquad \text{W-F1} = \sum_c \frac{n_c}{N}\,\mathrm{F1}_c. \tag{3}$$

W-F1 is a support-weighted average of per-class F1, so the three large classes (84% of dev) dominate it and a weak minority class costs relatively little. Over the set $\mathcal{R}$ of 277 reversible dev pairs, with gold labels (y_{12}, y_{21}) and predictions $(\hat{y}_{12}, \hat{y}_{21})$,

$$\text{SoftCons} = \frac{1}{|\mathcal{R}|}\sum_{\mathcal{R}} \mathbf{1}\big[\hat{y}_{12} \neq \text{NEG} \ \wedge \ \hat{y}_{21} = \mathrm{rev}(\hat{y}_{12})\big], \tag{4}$$

$$\text{HardCons} = \frac{1}{|\mathcal{R}|}\sum_{\mathcal{R}} \mathbf{1}\big[\hat{y}_{12} = y_{12} \ \wedge \ \hat{y}_{21} = y_{21}\big]. \tag{5}$$

SoftCons ignores gold labels, so a wrong system can score high on it. HardCons requires both orientations to be correct, so it is never above SoftCons when gold is coherent. Both use only reversible pairs, which in dev are all non-NEG, so single rows affect only W-F1. All three average over items, not classes.

Secondary metrics. Accuracy; Macro-F1, which weights the four classes equally and so is sensitive to NEG; per-class precision, recall and F1; and the number of strictly one-sided NEG pairs, the error type that motivated the method.

Statistical reliability. We report mean and standard deviation over three seeds. To compare systems we use a paired cluster bootstrap: 2,000 resamples of the 383 dev `pair_id` groups, keeping both orientations together, giving a 95% interval and one-sided p value for the seed-averaged difference. We do not claim a difference whose interval includes zero.

Subgroups. We report accuracy by gold label, for rows with and without a partner, by pair length (total words, in thirds) and by word overlap (Jaccard, in thirds), to check whether the overall score hides weak spots.

7 Experimental Protocol

Table 4 lists the setup shared by every run. Data come from the organisers' release; we use the official train and dev partitions, and the test set is hidden until January 2027. The validation split is 10% of train, sampled by `pair_id` with a fixed seed (128 pairs and 48 singles, 304 rows) and identical for every run.

Hyperparameters and selection. We ran no hyperparameter search. Settings follow A1 and common defaults, and λ and w_{NEG} were fixed before training. After each epoch we compute argmax W-F1 on the validation split and keep the best epoch. Dev is never used for training or checkpoint selection.

Missing and duplicated instances. No rows or labels are missing. Seven dev rows repeat a training pair exactly (15 ignoring case), always with the same label. We keep them because the official evaluation does; they are 2.3% of dev.

Class imbalance. NEG weighting in M3, and Macro-F1 and per-class recall in reporting.

Leakage prevention. The validation split is grouped by `pair_id`, so a pair and its reverse never straddle it. Train and dev share no `pair_id`. The decoding rules use

ID	Ablation	Hypothesis	Expected interpretation / preliminary outcome
A	RoBERTa (M0) vs DeBERTa-v3 (M1)	A stronger encoder improves the EQ/NEG boundary	Gain isolates the encoder. *Done:* +6.3 W-F1, +7.9 HardCons, both significant
B	Consistency loss on/off (M1 vs M2)	Training-time agreement raises HardCons	Gain isolates the loss. *Done:* no significant effect
C	$\lambda = 1$ vs $\lambda = 0.5$	The loss is not over- or under-weighted	Sensitivity of B to its weight. *Done, one seed:* no improvement
D	NEG weighting on/off (M2 vs M3)	Weighting raises NEG recall without hurting other classes	Inspect NEG precision/recall. *Done:* recall up, W-F1 change not significant
E	Decoding rule: argmax / symmetric / struct_pair / struct_full	Joint decisions remove one-sided errors; structure decides NEG	Separates model gains from inference gains. *Done:* both significant
F	Single seed vs three-seed ensemble	Averaging reduces seed variance	*Done:* +1.1 W-F1, significant
G	Frozen encoder (linear probe) vs fully fine-tuned	Task adaptation is needed beyond generic representations	Accuracy gain against training cost
H	Generic vs NLI-pretrained initialisation (MNLI-style checkpoint)	Prior entailment training helps the EQ/FWD/BWD decision	Gain on paired rows
I	Four-class vs three-class model on paired rows only	When NEG is decided structurally, the model need not learn it	Gain on EQ/FWD/BWD; removes "NEG both ways"
J	Checkpoint selection under argmax vs under the final decoding rule	Selection should match the deployed rule	Different epochs selected; effect on dev
K	Averaged vs joint pair decoding (best of 3 valid labelings)	Exact pair-level decoding beats averaging	Change in HardCons
L	Base vs large encoder; same- vs cross-backbone ensembles	Capacity and diversity help	Gain against cost

Table 3: Ablation plan. Significance is judged with the paired cluster bootstrap of Section 6; preliminary outcomes are detailed in Section 9.

Setting	Value
Data	Track 1 (EN), organisers' release, commit `588968e` (8 Sep 2026)
Partitions	train 3,042; dev 660 (277 pairs + 106 singles); test hidden
Validation	10% of train by `pair_id` (304 rows), seed 2027, same for all runs
Tokenization	cased model tokenizer; pair input; max 64 tokens
Initialisation	pretrained weights; random head
Optimiser	AdamW, lr 2×10^{-5}, weight decay 0.01
Schedule	10% linear warm-up, linear decay
Batch	8 pairs (≈16 rows)
Regularisation	dropout 0.1; grad. clip 1.0
Epochs / selection	6; best validation argmax W-F1
Loss settings	$\lambda \in \{0, 0.5, 1\}$, $w_{\text{NEG}} \in \{1, 2\}$
Precision	fp32
Seeds	13, 42, 87
Hardware	Colab NVIDIA T4 (16 GB)
Software	Python 3.13, PyTorch 2.14.1 (CUDA 13.0), Transformers 5.22.1, scikit-learn 1.6.1 (Colab)
Cost	2 min (RoBERTa), 3.3 min (DeBERTa) per run

Table 4: Training and evaluation configuration, shared by all A2 runs.

`pair_id` from the participant file and never gold labels. The test set is unseen.

Local versus server evaluation. Our local scorer reproduces the Codabench development score exactly; hidden-test scores will come only from the server. Dev was not used for selection, but we did use it to compare decoding rules and configurations, so dev scores are not an independent estimate of test performance.

8 Initial Implementation

The system runs end to end in PyTorch and Hugging Face Transformers, driven by one Colab notebook (Table 5).

We checked each of the brief's minimum requirements.

Brief requirement	Implementation
Data loading	`src/data.py` (official CSVs, labels)
Representation	`PairCollator`: both orientations of a pair, tokenized together
Model, training	`src/train_a2.py`: `run(**config)`, loss, validation selection
Inference, decoding	`predict_probs`; `src/decoding.py` (four rules)
Evaluation	official scorer; `06_decoding_experiments.py`; `07_a2_analysis.py`
Configuration	defaults in `train_a2.py`; full config per run in `runs/<tag>.json`
Checkpoints, seeds	`save/load_checkpoint`; `set_seed`

Table 5: Implementation components.

Row counts, label counts and the pair structure match the release, and each batch holds both orientations of its pairs. All 13 runs finished with finite losses and produced predictions in the Codabench format, and our metric code matches the official scorer exactly. Retraining M3 at seed 42 from its stored configuration reproduced all 660 dev labels of the original run (W-F1 0.829, HardCons 0.820). We saved that checkpoint (about 0.74 GB), reloaded it in a separate step, and got the same 660 labels and scores.

9 Preliminary Results

These runs verify the pipeline and are not final performance claims. We trained on 2,738 rows, validated on 304 and evaluated on all 660 dev rows, with 13 runs (M0 to M3 at three seeds, M2b at one) taking about 40 GPU-minutes on a Colab T4: about 2 minutes per RoBERTa run and 3.3 per DeBERTa run.

Protocol check. M0 reproduces the A1 RoBERTa result (0.758 against 0.765, within seed variation), so the new protocol does not distort the comparison with A1.

	argmax		symmetric		struct_pair		struct_full	
System	W-F1	HardCons	W-F1	HardCons	W-F1	HardCons	W-F1	HardCons
M0 RoBERTa	0.758 ±.026	0.759 ±.025	0.768 ±.034	0.821 ±.025	0.803 ±.036	0.869 ±.026	0.889 ±.022	0.869 ±.026
M1 DeBERTa-v3	0.821 ±.006	**0.839** ±.006	0.821 ±.006	**0.862** ±.015	0.853 ±.010	0.904 ±.008	0.919 ±.006	0.904 ±.008
M2 + cons.	0.820 ±.005	0.827 ±.011	0.822 ±.005	0.845 ±.006	0.870 ±.013	0.909 ±.008	0.923 ±.007	0.909 ±.008
M3 + NEG wt.	**0.827** ±.004	0.828 ±.015	**0.824** ±.004	0.842 ±.006	**0.877** ±.010	**0.911** ±.004	**0.925** ±.004	**0.911** ±.004
M3, 3-seed ens.	0.832	0.830	0.837	0.852	0.893	0.924	0.936	0.924

Table 6: Dev results, mean ± std over seeds 13, 42 and 87 (official scorer). SoftCons is 1.0 for both structural rules by construction; under argmax it is 0.819, 0.892, 0.884 and 0.878 for M0–M3, and under symmetric 0.919, 0.931, 0.910 and 0.906. The last row is a single system (seed probabilities averaged), not a mean. Bold marks the best mean among M0–M3 in each column; differences among M1–M3 are not significant (Table 7).

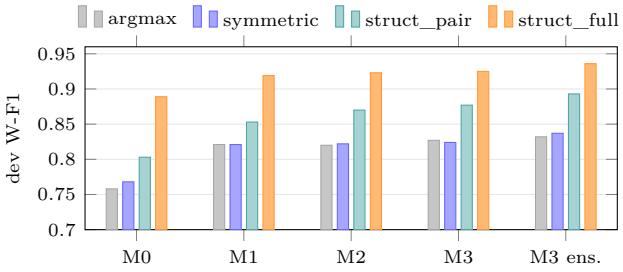


Figure 3: Dev W-F1 by system and decoding rule (mean of three seeds; "M3 ens." is the three-seed ensemble). The encoder change (M0 to M1) and the structural rules give the large gains; the training changes (M1 to M3) are within noise.

Comparison	ΔW-F1 [CI]	ΔHardCons [CI]
A: M1 vs M0, argmax	+6.3 [+3.0, +9.7]*	+7.9 [+3.7, +11.9]*
B: M2 vs M1, argmax	−0.1 [−1.5, +1.2]	−1.2 [−2.8, +0.4]
B: M2 vs M1, s_f.	+0.4 [−1.0, +1.7]	+0.5 [−1.2, +2.0]
D: M3 vs M2, argmax	+0.7 [−0.5, +1.9]	+0.1 [−1.3, +1.6]
D: M3 vs M2, s_f.	+0.2 [−0.7, +1.2]	+0.2 [−0.8, +1.5]
E: M1 symmetric vs argmax	0.0 [−0.6, +0.5]	+2.3 [+1.3, +3.4]*
E: M1 s_f. vs argmax	+9.8 [+7.6, +12.3]*	+6.5 [+4.4, +8.8]*
F: M3 ens. vs seeds, s_f.	+1.1 [+0.3, +1.9]*	+1.3 [+0.4, +2.3]*

Table 7: Paired cluster bootstrap over dev `pair_id` groups (2,000 resamples), in percentage points. *: 95% interval excludes 0 (one-sided $p < 0.01$). Letters refer to Table 3; s_f. = struct_full.

Group	n	M0 argmax	M1 argmax	M3 argmax	M3 s_full
Gold EQ	174	0.695	0.839	0.797	0.851
Gold FWD	190	0.870	0.868	0.867	0.939
Gold BWD	190	0.851	0.875	0.868	0.939
Gold NEG	106	0.506	0.610	0.708	1.000
With partner	554	0.809	0.862	0.845	0.911
Without partner	106	0.506	0.610	0.708	1.000
Short (≤5 words)	312	0.794	0.855	0.862	0.949
Medium (6–7)	210	0.743	0.829	0.837	0.914
Long (>7)	138	0.710	0.734	0.715	0.889
Low overlap (J≤.25)	250	0.688	0.779	0.773	0.909
Mid overlap	299	0.778	0.843	0.846	0.938
High overlap (J>.5)	111	0.874	0.859	0.874	0.928

Table 8: Dev accuracy by gold label (i.e. recall) and by subgroup, mean of three seeds. Length is total words in the pair; J is word Jaccard overlap.

Encoder (ablation A). DeBERTa-v3 adds 6.3 W-F1 and 7.9 HardCons under argmax, both significant (Table 7), and its seed standard deviations are about four times smaller than RoBERTa's (Table 6). Its gains are largest on the two hardest classes: EQ recall rises from 0.695 to 0.839 and NEG recall from 0.506 to 0.610 (Table 8). Training in fp32 removed the NaN losses seen on Apple MPS in A1.

Decoding (ablation E). Symmetric decoding adds 2.3 HardCons points for M1, with W-F1 unchanged, and removes every one-sided NEG pair. The structural rules add 9.8 W-F1 (Figure 3): every NEG row becomes correct, and EQ/FWD/BWD rows previously lost to NEG are recovered (Table 9). This gain comes from how the dataset was built, not from better language understanding. Applied to the saved A1 models, the same rules lift RoBERTa from 0.765 to 0.874 and BERT from 0.705 to 0.838.

	argmax				struct_full			
Gold	EQ	FWD	BWD	NEG	EQ	FWD	BWD	NEG
EQ	**137**	7	6	24	**150**	12	12	0
FWD	7	**165**	1	17	7	**181**	2	0
BWD	8	2	**166**	14	7	2	**181**	0
NEG	13	7	8	**78**	0	0	0	**106**

Table 9: Confusion matrices of the M3 ensemble on dev (rows: gold, with 174 EQ, 190 FWD, 190 BWD, 106 NEG; columns: predicted).

Consistency loss (ablations B and C). We found no significant effect. Under argmax, M2 has 1.2 points lower HardCons than M1 (interval −2.8 to +0.4), while its one-sided NEG count falls from 9.0 to 6.7 per seed. The model satisfies the loss by predicting NEG in both orientations: pairs labelled NEG both ways rise from 14.7 to 21.7 per seed, which removes one-sided errors without fixing either label. Under struct_full the difference (+0.4 W-F1) spans zero, and $\lambda = 0.5$ was no better. This fits A1: the loss enforces directional agreement, but the remaining errors are mostly about NEG.

Neg weighting (ablation D). NEG recall rises from 0.61 to 0.71 and precision falls from 0.64 to 0.59, giving the best NEG F1 (0.643) and Macro-F1 (0.806), but the W-F1 change over M2 is not significant. We call M3 the best configuration tested, without claiming each component helps.

Ensembling (ablation F). Averaging three seeds adds 1.1 W-F1 and 1.3 HardCons, both significant. The M3 ensemble with struct_full reaches 0.936 W-F1 and 0.924 HardCons (last row of Table 6). Submitted to the Codabench development phase (English only), it scored identically to our local scorer and ranked 7th of 26 English entries on 8 November. This confirms the format and scorer, not test performance.

Subgroups. Rows without a partner (all NEG) are much harder than paired rows under argmax (0.61 against 0.86 for M1). Long pairs are the hardest group and gain least from the stronger encoder (+2.4 accuracy against +6.1 for short pairs), while low-overlap pairs gain most (+9.1), since word matching cannot solve them (Table 8).

Evidence of meaningful predictions. The confu-

sion matrices are strongly diagonal, and FWD and BWD are almost never swapped (3 to 4 rows), so the model has learned direction and is not relying on length. Under struct_full, 21 pairs remain wrong, all on EQ versus entailment.

Warnings and failures. Three seed-87 result files did not reach Drive after a Colab reset; we regenerated them from the saved probabilities with identical scores. A1's count of 42 "one-sided" NEG pairs also included pairs predicted NEG both ways; the strict count is 15 to 19 per seed. M1's struct_full W-F1 is identical for seeds 13 and 42, but the seeds disagree on 24 rows and each gets 12 of them right, so this is a coincidence and not a duplicated run.

Issues to resolve. Before the full experiments we need to confirm that the test file keeps the pair structure, match checkpoint selection to the decoding rule, and redesign the consistency loss so that NEG in both orientations cannot satisfy it.

10 Reproducibility and Limitations

Reproducibility. The repository[1] holds the exact code, each run's configuration, history and probabilities, and a manifest linking every reported number to its files. All scores were recomputed from the saved probabilities with the official scorer and match the original runs. Every system used the same split, preprocessing and scoring scripts. Seeds are fixed and the M3 seed-42 rerun reproduced every label, but GPU kernels are not guaranteed to be deterministic, so we report seed means and standard deviations.

Limitations. The structural rules rely on a property of train and dev that may not hold on test, so we will report test results with and without them. Dev informed our configuration choices, so dev scores are optimistic. Three seeds are few. Checkpoints are selected under argmax although the best results use structured decoding. No hyperparameter search was run, only English is covered, and we have not yet measured how many gold labels are debatable.

11 Planned Experiments

Under the structural rules, every remaining error is an EQ/FWD/BWD decision on a paired row (21 of 277 pairs). Assignment 3 will target that decision and run ablations G to L (Table 3):

- a frozen versus fine-tuned encoder, to measure what task adaptation adds;
- a DeBERTa-v3 checkpoint already trained on NLI data;
- a three-class model trained on paired rows only, which also removes the "NEG both ways" solution;
- checkpoint selection under the final decoding rule;
- joint decoding over the three valid pair labelings (EQ,EQ), (FWD,BWD) and (BWD,FWD);
- DeBERTa-v3-large and cross-backbone ensembles (Lakshminarayanan et al., 2017).

We will also add seeds, run a hyperparameter search on the validation split, audit the remaining errors and their gold labels, run the multilingual pilot for Spanish, Basque and mixed pairs, and submit to the January 2027 test phase with and without the structural rules.

[1] https://github.com/MuneebBaig71/DiCo-NLI_SemEval

References

Jon Felix Apaolaza Larraya, Begoña Altuna, Aitor Soroa, and Inigo Lopez-Gazpio. Assessing logical coherence of LLMs via fine-grained NLI. In *Proceedings of the Fifteenth Language Resources and Evaluation Conference (LREC 2026)*, pages 5431–5444, Palma, Mallorca, Spain, 2026. European Language Resources Association. doi: 10.63317/4prei82n6ev9.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1*, pages 4171–4186. Association for Computational Linguistics, 2019. doi: 10.18653/v1/N19-1423.

DiCo-NLI Organizers. SemEval 2027 task 2: Directional-consistent fine-grained natural language inference (DiCo-NLI). https://github.com/ilopezgazpio/SemEval-2027-Task-2-DiCo-NLI, 2026. Accessed 4 October 2026.

Suchin Gururangan, Swabha Swayamdipta, Omer Levy, Roy Schwartz, Samuel R. Bowman, and Noah A. Smith. Annotation artifacts in natural language inference data. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2*, pages 107–112. Association for Computational Linguistics, 2018. doi: 10.18653/v1/N18-2017.

Pengcheng He, Jianfeng Gao, and Weizhu Chen. DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing. In *The Eleventh International Conference on Learning Representations*, 2023. URL https://arxiv.org/abs/2111.09543.

Myeongjun Jang, Deuk Sin Kwon, and Thomas Lukasiewicz. BECEL: Benchmark for consistency evaluation of language models. In *Proceedings of the 29th International Conference on Computational Linguistics*, pages 3680–3696, Gyeongju, Republic of Korea, 2022. International Committee on Computational Linguistics. URL https://aclanthology.org/2022.coling-1.324/.

Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL https://arxiv.org/abs/1612.01474.

Tao Li, Vivek Gupta, Maitrey Mehta, and Vivek Srikumar. A logic-driven framework for consistency of neural models. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, pages 3924–3935, 2019. doi: 10.18653/v1/D19-1405.

Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. RoBERTa: A robustly optimized BERT pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.

Bill MacCartney. *Natural Language Inference*. PhD thesis, Stanford University, 2009.

Pasquale Minervini and Sebastian Riedel. Adversarially regularising neural NLI models to integrate logical background knowledge. In *Proceedings of the 22nd Conference on Computational Natural Language Learning*, pages 65–74, 2018. doi: 10.18653/v1/K18-1007.